

Table of Content

S.No	Topic	Date	Page No.	Sign
1	Plotting of Basic Signals using MATLAB			
2	Analog-to-Digital Conversion (Sampling, Quantization and Encoding) using MATLAB			
3	Implementation of Huffman Coding for Lossless Data Compression			
4	Implementation of Shannon-Fano Coding for Lossless Data Compression			
5	Simulation of Pulse Code Modulation (PCM) using MATLAB and Simulink			
6	Simulation of Differential Pulse Code Modulation (DPCM) using MATLAB and Simulink			
7	Simulation of Delta Modulation (DM) using MATLAB and Simulink			
8	Simulation of Adaptive Delta Modulation (ADM) using MATLAB and Simulink			

EXPERIMENT 1

Objective:

To plot basic signals (Unit Step, Ramp, Impulse, Sinusoidal, Exponential and Square Wave) using MATLAB.

Prerequisites:

- Set up MATLAB software.

Theory:

In signal analysis, a signal is a function that represents the variation of a physical quantity with respect to time or space. Signals are widely used in electrical engineering, communication systems, and electromagnetic analysis to represent quantities such as voltage, current, or field intensity. Basic signals serve as fundamental building blocks for understanding complex systems and signal processing techniques.

Using MATLAB, these signals can be generated mathematically and visualised through plots, which helps in studying their behaviour and characteristics.

1. Unit Step Signal

The unit step signal is a basic signal that changes its value from zero to one at a particular instant of time. It is widely used to represent the sudden switching of a system.

$$u(t) = 0, \text{ for } t < 0; \quad u(t) = 1, \text{ for } t \geq 0$$

2. Ramp Signal

The ramp signal increases linearly with time. It represents a signal whose magnitude continuously grows with time.

$$r(t) = 0, \text{ for } t < 0; \quad r(t) = t, \text{ for } t \geq 0$$

3. Impulse Signal

The unit impulse signal is an ideal signal that has zero value everywhere except at one instant, where its value is infinitely large but its total area is equal to one. It is represented as $\delta(t)$, with the important property:

$$\int_{-\infty}^{\infty} \delta(t) dt = 1$$

The impulse signal is useful in analysing system responses because it can excite all frequencies of a system simultaneously.

4. Sinusoidal Signal

A sinusoidal signal is a periodic signal commonly used in electrical and communication systems. It represents oscillatory behaviour such as alternating current or electromagnetic waves, and can be given in general as:

$$x(t) = A \sin(\omega t + \phi)$$

- A = amplitude
- ω = angular frequency
- ϕ = phase angle
- t = time

5. Exponential Signal

An exponential signal represents signals that grow or decay exponentially with time:

$$x(t) = A e^{at}$$

If $a < 0$, the signal decays with time. If $a > 0$, the signal increases with time. Exponential signals are commonly used to represent transient responses in electrical circuits.

6. Square Wave Signal

A square wave signal is a periodic signal that alternates between two fixed levels, usually positive and negative values. It is widely used in digital electronics and communication systems, and can be represented as a periodic switching function:

$$sq(t) = A, \text{ for } 0 < t < T/2; \quad sq(t) = -A, \text{ for } T/2 < t < T$$

MATLAB

```
% Time vector for continuous signals
t_cont= -5:0.01:5;
% Discrete time vector for impulse signal
t_disc= -5:1:5;

% 1. Unit Step Signal
subplot(3, 2, 1);
u = t_cont >= 0; % Unit step signal
plot(t_cont, u, "LineWidth", 1.5);
xlabel('Time'); ylabel('Amplitude');
title('Unit Step Signal'); grid on;

% 2. Ramp Signal
subplot(3, 2, 2);
r = t_cont .* (t_cont >= 0); % Ramp signal
plot(t_cont, r, "LineWidth", 1.5);
xlabel("Time"); ylabel("Amplitude");
title("Ramp Signal"); grid on;

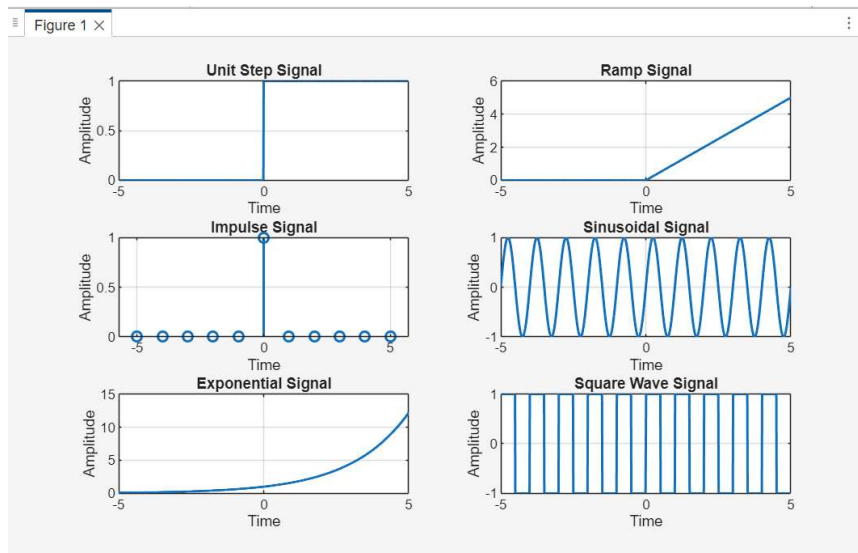
% 3. Impulse Signal
subplot(3, 2, 3);
impulse = t_disc == 0; % Impulse Signal
stem(t_disc, impulse, "LineWidth", 1.5);
xlabel("Time"); ylabel("Amplitude");
title("Impulse Signal"); grid on;

% 4. Sinusoidal Signal
subplot(3, 2, 4);
f = 1; a = 1; phi = 0;
y_sin = a * sin(2*pi*f*t_cont + phi);
plot(t_cont, y_sin, "LineWidth", 1.5);
xlabel("Time"); ylabel("Amplitude");
title("Sinusoidal Signal"); grid on;

% 5. Exponential Signal
subplot(3, 2, 5);
alpha = 0.5; % Exponential Growth
y_exp = a * exp(alpha * t_cont);
plot(t_cont, y_exp, "LineWidth", 1.5);
xlabel("Time"); ylabel("Amplitude");
title("Exponential Signal"); grid on;

% 6. Square Wave Signal
subplot(3, 2, 6);
sq = sign(sin(2*pi*t_cont)); % Square wave signal
plot(t_cont, sq, "LineWidth", 1.5);
xlabel('Time'); ylabel('Amplitude');
title('Square Wave Signal'); grid on;
```

Output:



Observation:

- The Unit Step signal jumps from 0 to 1 at $t = 0$, while the Ramp signal increases linearly for $t \geq 0$.
- The Impulse signal has a non-zero value only at $t = 0$, confirming its ideal, instantaneous nature.
- The Sinusoidal signal is periodic and oscillates smoothly between -1 and 1, whereas the Exponential signal grows continuously with time for $a > 0$.
- The Square Wave signal alternates sharply between -1 and 1, representing a typical digital switching waveform.

Result:

All six basic signals — Unit Step, Ramp, Impulse, Sinusoidal, Exponential and Square Wave — were successfully generated and plotted using MATLAB, and their distinct characteristics were verified graphically.

EXPERIMENT 2

Objective:

To perform the conversion of an analog signal into a digital signal (sampling, quantization and encoding) using MATLAB.

Prerequisites:

- Set up MATLAB software.
- Basic knowledge of sampling theorem and quantization.

Theory:

Analog signals are continuous-time signals that can take any value within a given range. In contrast, digital signals are represented using discrete values, usually in the form of binary numbers. Analog-to-Digital Conversion (ADC) is the process of converting a continuous analog signal into a digital representation that can be processed and stored by a digital system.

The conversion of an analog signal into a digital signal generally involves three main steps:

- **Sampling:** The continuous-time analog signal is sampled at discrete time intervals. According to the Nyquist sampling theorem, the sampling frequency f_s must be at least twice the highest frequency f_m present in the signal, i.e. $f_s \geq 2f_m$. This prevents aliasing and allows the original signal to be reconstructed.
- **Quantization:** The sampled amplitudes are rounded to the nearest available discrete level. If an n -bit ADC is used, the number of quantization levels is $L = 2^n$. A higher number of bits provides more quantization levels and hence better amplitude resolution.
- **Encoding:** Each quantized amplitude level is represented by a corresponding binary code. The resulting sequence of binary values constitutes the digital representation of the original analog signal.

MATLAB can be used to simulate this conversion by generating an analog signal, sampling it at a specified sampling frequency, quantizing the samples, and representing the quantized values digitally. Thus, analog-to-digital conversion allows continuous analog information to be represented in a form suitable for digital processing, storage, and transmission.

MATLAB

```
% Continuous signal
A = 5; f = 2;
t = 0:0.001:2;
x = A*sin(2*pi*f*t);

% Sampling
Fs = 20;
ts = 0:1/Fs:2;
xs = A*sin(2*pi*f*ts);

% Quantization
L = 8; % 8 levels
xmin = -A; xmax = A;
delta = (xmax-xmin)/(L-1);
index = round((xs-xmin)/delta);
index(index<0) = 0;
index(index>L-1) = L-1;
xq = xmin + index*delta;

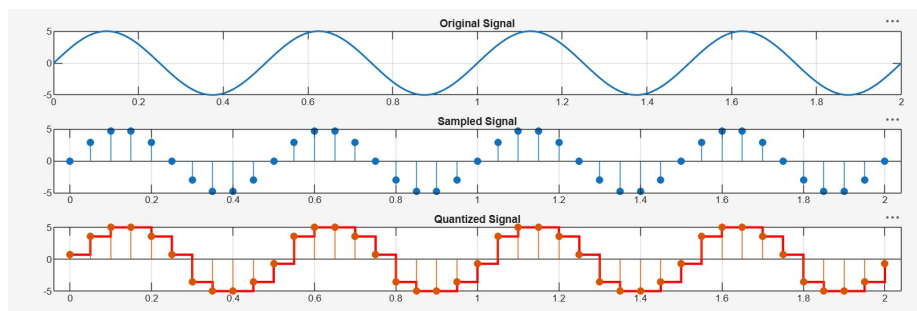
% Encoding
bits = ceil(log2(L));
disp('Sample   Quantized Value   Binary Code')
for i = 1:length(index)
    code = dec2bin(index(i), bits);
    fprintf('%2d\t %8.2f\t\t\t%s\n', i, xq(i), code);
end
```

```
% Plot
figure
subplot(3,1,1); plot(t,x,'LineWidth',1.5); title('Original Signal'); grid on
subplot(3,1,2); stem(ts,xs,'filled'); title('Sampled Signal'); grid on
subplot(3,1,3); stairs(ts,xq,'r','LineWidth',2); hold on
stem(ts,xq,'filled'); title('Quantized Signal'); grid on
```

Output:

OUTPUT

Sample	Quantized Value	Binary Code
1	0.71	100
2	3.57	110
3	5.00	111
4	5.00	111
5	3.57	110
6	0.71	100
7	-3.57	001
8	-5.00	000
9	-5.00	000
10	-3.57	001
11	-0.71	011
12	3.57	110
13	5.00	111
14	5.00	111
15	3.57	110
16	0.71	100



Observation:

- The sampled signal correctly captures the amplitude of the original sinusoid at regular time intervals of $1/F_s$.
- The quantized signal approximates the sampled values using one of the 8 available discrete levels, resulting in a small quantization error.
- Each quantized level is uniquely represented by a 3-bit binary code, as $L = 8 = 2^3$.

Result:

The analog sinusoidal signal was successfully sampled, quantized and encoded into binary form using MATLAB, and the original, sampled and quantized signals were plotted for comparison.

EXPERIMENT 3

Objective:

To implement the Huffman coding algorithm for lossless data compression using MATLAB.

Prerequisites:

- Set up MATLAB software.
- Basic knowledge of probability and binary trees.

Theory:

Huffman coding is a lossless data compression technique used to represent data using variable-length binary codes. It is based on the frequency or probability of occurrence of each symbol in the input data. Symbols that occur more frequently are assigned shorter codes, while less frequent symbols are assigned longer codes. The main objective of Huffman coding is to reduce the average number of bits required to represent the data without losing any information.

Principle of Huffman Coding

Huffman coding follows a greedy approach. The two symbols having the lowest frequencies are repeatedly combined to form a new node. This process continues until a single binary tree, called the Huffman tree, is formed. The procedure is:

- Determine the frequency or probability of occurrence of each symbol.
- Arrange the symbols in increasing order of their frequencies.
- Select the two symbols with the lowest frequencies and combine them into a new node.
- Repeat the process until only one node remains, forming the Huffman tree.
- Assign binary values, generally 0 and 1, to the branches of the tree.
- Obtain the Huffman code for each symbol by following the path from the root to that symbol.
- Replace each symbol in the original data with its corresponding Huffman code.

Properties of Huffman Coding

- It is a lossless compression technique, so the original data can be recovered exactly.
- It uses variable-length codes.
- No Huffman code is a prefix of another code, making the code uniquely decodable.
- Symbols with higher frequencies generally receive shorter codes.
- It is widely used in data compression and forms a component of techniques such as JPEG and DEFLATE.

MATLAB

```
clc; clear;
symbols = {'A','B','C','D','E'};
prob = [0.40 0.25 0.15 0.10 0.10];
codes = {'',' ',' ',' ',' '};
nodes = symbols;
p = prob;

while length(p) > 1
    [p,idx] = sort(p);
    nodes = nodes(idx);
    for i = 1:5
        if contains(nodes{1},symbols{i})
            codes{i} = ['0' codes{i}];
        elseif contains(nodes{2},symbols{i})
            codes{i} = ['1' codes{i}];
        end
    end
end
```

```

        nodes{1} = [nodes{1} nodes{2}];
        p(1) = p(1) + p(2);
        nodes(2) = [];
        p(2) = [];
    end

    fprintf('Symbol\tProbability\tCode\n');
    for i = 1:5
        fprintf('%s\t%.2f\t\t%s\n', symbols{i}, prob(i), codes{i});
    end

    H = -sum(prob .* log2(prob));
    Lavg = sum(prob .* cellfun(@length, codes));
    eff = H/Lavg * 100;
    fprintf('\nEntropy = %.4f bits/symbol\n', H);
    fprintf('Average Code Length = %.4f bits/symbol\n', Lavg);
    fprintf('Efficiency = %.2f%%\n', eff);

```

Output:

OUTPUT

Symbol	Probability	Code
A	0.40	0
B	0.25	10
C	0.15	110
D	0.10	1110
E	0.10	1111

Entropy = 2.1464 bits/symbol
 Average Code Length = 2.1500 bits/symbol
 Efficiency = 99.83%

Observation:

- The most probable symbol 'A' (probability 0.40) received the shortest code (0), while the least probable symbols 'D' and 'E' received the longest codes (4 bits each).
- The average code length (2.15 bits/symbol) is very close to the source entropy (2.1464 bits/symbol), indicating that the coding scheme is nearly optimal.
- The resulting coding efficiency of 99.83% shows that Huffman coding compresses the data close to the theoretical limit set by entropy.

Result:

The Huffman coding algorithm was successfully implemented in MATLAB. The Huffman codes were generated for the given symbols based on their probabilities, and the entropy, average code length, and coding efficiency were calculated, demonstrating efficient lossless data compression.

EXPERIMENT 4

Objective:

To implement the Shannon-Fano encoding algorithm for lossless data compression using MATLAB.

Prerequisites:

- Set up MATLAB software.
- Basic knowledge of probability and prefix coding.

Theory:

Shannon-Fano coding is a lossless data compression technique that represents symbols using variable-length binary codes. Symbols with higher probabilities are assigned shorter codes, while symbols with lower probabilities are assigned longer codes.

The symbols are first arranged in descending order of probability. They are then divided into two groups with nearly equal total probabilities. The first group is assigned 0 and the second group 1. This process is repeated for each group until every symbol has a unique code. Shannon-Fano coding is a prefix coding technique, so the resulting codes can be decoded without ambiguity. Its performance can be evaluated using entropy, average code length, and coding efficiency.

Algorithm

- Enter the symbols and their probabilities.
- Arrange the symbols in descending order of probability.
- Divide the symbols into two groups with nearly equal total probabilities.
- Assign 0 to one group and 1 to the other.
- Repeat the division until each group contains one symbol.
- Obtain the binary code for each symbol.
- Calculate the entropy, average code length, and coding efficiency.

MATLAB

```
function codes = shannonFano(symbols, prob)
    [prob, idx] = sort(prob, 'descend');
    symbols = symbols(idx);
    codes = repmat({''}, 1, length(symbols));
    codes = sf(symbols, prob, codes, 1:length(symbols));
end

function codes = sf(symbols, prob, codes, ind)
    if length(ind) <= 1
        return;
    end
    total = sum(prob(ind));
    s = 0; split = 1;
    for i = 1:length(ind)-1
        s = s + prob(ind(i));
        if abs(total/2 - s) < abs(total/2 - (s-prob(ind(i))))
            split = i;
        end
    end
    codes(ind(1:split)) = cellfun(@(x) ['0' x], ...
        codes(ind(1:split)), 'UniformOutput', false);
    codes(ind(split+1:end)) = cellfun(@(x) ['1' x], ...
        codes(ind(split+1:end)), 'UniformOutput', false);
    codes = sf(symbols, prob, codes, ind(1:split));
    codes = sf(symbols, prob, codes, ind(split+1:end));
end

% Example
```

```

symbols = {'A','B','C','D'};
prob = [0.4 0.3 0.2 0.1];
codes = shannonFano(symbols, prob);

fprintf('Shannon-Fano Codes:\n');
fprintf('Symbol\tProbability\tCode\n');
for i = 1:length(symbols)
    fprintf('%s\t%.2f\t\t%s\n', symbols{i}, prob(i), codes{i});
end

H = -sum(prob .* log2(prob));
Lavg = sum(prob .* cellfun(@length,codes));
eff = (H/Lavg)*100;
fprintf('\nEntropy = %.4f bits/symbol\n', H);
fprintf('Average Code Length = %.4f bits/symbol\n', Lavg);
fprintf('Efficiency = %.2f%%\n', eff);

```

Output:

OUTPUT

```

Shannon-Fano Codes:
Symbol  Probability  Code
A       0.40       0
B       0.30      01
C       0.20      011
D       0.10      111

```

```

Entropy = 1.8464 bits/symbol
Average Code Length = 1.9000 bits/symbol
Efficiency = 97.18%

```

Observation:

- The symbol 'A' with the highest probability (0.40) was assigned the shortest code, while 'D' with the lowest probability (0.10) received the longest code.
- The average code length (1.90 bits/symbol) is slightly greater than the entropy (1.8464 bits/symbol), which is expected since Shannon-Fano coding is not always optimal, unlike Huffman coding.
- The resulting coding efficiency of 97.18% confirms that Shannon-Fano coding still provides good, though slightly sub-optimal, compression.

Result:

The Shannon-Fano coding technique was successfully implemented to generate binary codes for the given symbols based on their probabilities. The symbols with higher probabilities were assigned shorter codes, while symbols with lower probabilities were assigned longer codes. The average code length and coding efficiency were calculated, demonstrating the effectiveness of Shannon-Fano coding for lossless data compression.

EXPERIMENT 5

Objective:

To simulate and implement Pulse Code Modulation (PCM) using MATLAB and Simulink.

Prerequisites:

- Set up MATLAB software.
- Set up Simulink.
- Basic knowledge of sampling and quantization theory.

Theory:

Pulse Code Modulation (PCM) is a method used to digitally represent sampled analog signals. It is the most fundamental form of digital communication and forms the basis for more advanced techniques such as DPCM and Delta Modulation.

The conversion of an analog signal into a digital PCM signal generally involves three main steps:

- **Sampling:** The continuous-time analog signal is sampled at discrete time intervals. According to the Nyquist sampling theorem, the sampling frequency f_s must be at least twice the highest frequency f_m present in the signal, i.e. $f_s \geq 2f_m$, to prevent aliasing.
- **Quantization:** The sampled amplitudes are rounded to the nearest available discrete level. If an n-bit ADC is used, the number of quantization levels is $L = 2^n$.
- **Encoding:** Each quantized amplitude level is represented by a corresponding binary code, forming the digital PCM output.

In MATLAB, the PCM process can be simulated by performing sampling, quantization, and binary encoding of an input signal. Simulink can be used to represent the same PCM system graphically using interconnected blocks for sampling, quantization, and encoding.

MATLAB

```
% Continuous signal
A = 5; f = 2;
t = 0:0.001:2;
x = A*sin(2*pi*f*t);

% Sampling
Fs = 20;
ts = 0:1/Fs:2;
xs = A*sin(2*pi*f*ts);

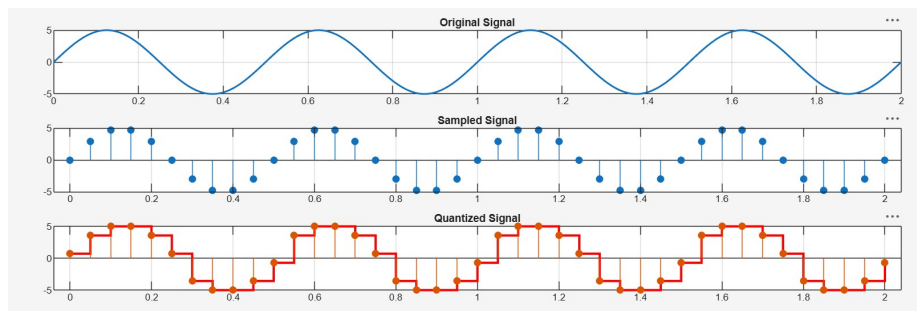
% Quantization
L = 8; % 8 levels
xmin = -A; xmax = A;
delta = (xmax-xmin)/(L-1);
index = round((xs-xmin)/delta);
index(index<0) = 0;
index(index>L-1) = L-1;
xq = xmin + index*delta;

% Encoding
bits = ceil(log2(L));
disp('Sample   Quantized Value   Binary Code')
for i = 1:length(index)
    code = dec2bin(index(i), bits);
    fprintf('%2d\t %8.2f\t\t\t%s\n', i, xq(i), code);
end

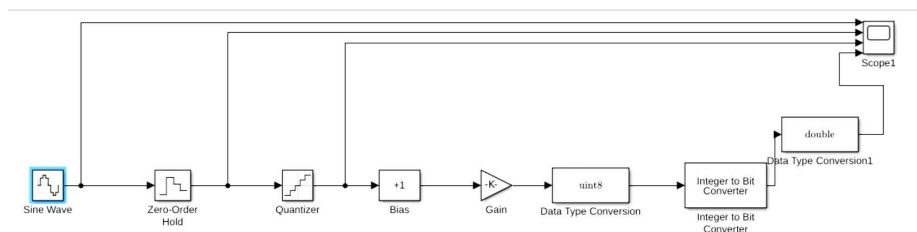
% Plot
figure
subplot(3,1,1); plot(t,x,'LineWidth',1.5); title('Original Signal'); grid on
```

```
subplot(3,1,2); stem(ts,xs,'filled'); title('Sampled Signal'); grid on
subplot(3,1,3); stairs(ts,xq,'r','LineWidth',2); hold on
stem(ts,xq,'filled'); title('Quantized Signal'); grid on
```

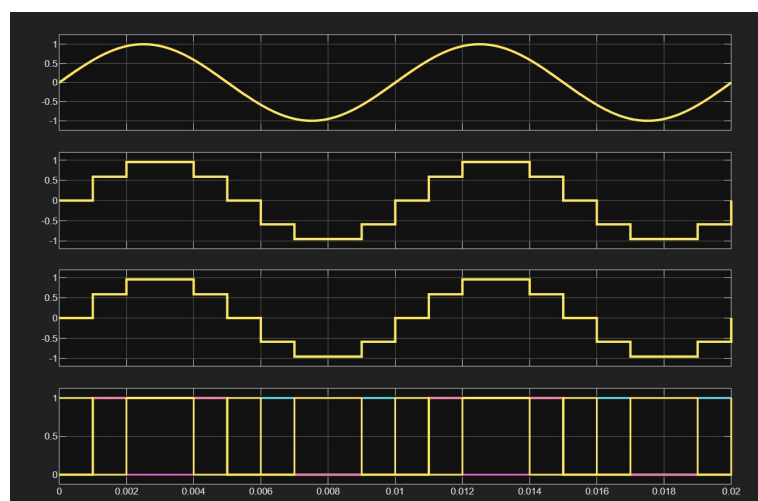
MATLAB Output:



Simulink Implementation:



Simulink Output:



Observation:

- The MATLAB simulation produces the same original, sampled and quantized waveforms as expected from a standard PCM system.
- The Simulink model uses a Sine Wave, Zero-Order Hold, Quantizer, Bias, Gain and Data Type Conversion blocks in sequence to replicate sampling, quantization and encoding.
- The Simulink scope output confirms that the quantized (PCM) waveform closely follows the shape of the original sine wave, while showing the expected staircase quantization pattern.

Result:

Pulse Code Modulation was successfully simulated and implemented using both MATLAB and Simulink, and the sampled and quantized waveforms obtained from both approaches were found to be consistent with each other.

EXPERIMENT 6

Objective:

To simulate and implement Differential Pulse Code Modulation (DPCM) using MATLAB and Simulink.

Prerequisites:

- Set up MATLAB software.
- Set up Simulink.
- Basic knowledge of PCM and prediction-based coding.

Theory:

Differential Pulse Code Modulation (DPCM) is an improved form of Pulse Code Modulation (PCM) that encodes the difference between the present sample and its predicted value, rather than encoding the complete sample. Since adjacent samples of a slowly varying signal are highly correlated, this difference is generally smaller and can be represented using fewer bits.

In DPCM, the input signal is first sampled and compared with the predicted value produced by a predictor. The difference, called the prediction error, is then quantized and encoded into a digital bit sequence. At the receiver, the decoded prediction error is added to the predicted sample to reconstruct the original signal.

The basic DPCM system consists of a predictor, subtractor, quantizer, encoder, decoder, and adder. By transmitting only the prediction error, DPCM can reduce the required bit rate and bandwidth compared with conventional PCM while maintaining good signal reconstruction.

MATLAB

```
clc; clear; close all;

% Continuous signal
A = 5; f = 2;
t = 0:0.001:2;
x = A*sin(2*pi*f*t);

% Sampling
Fs = 20;
ts = 0:1/Fs:2;
xs = A*sin(2*pi*f*ts);

% DPCM Prediction
pred = [0 xs(1:end-1)];

% Difference signal
d = xs - pred;

% Quantization of difference
L = 8; % 8 levels
dmin = min(d); dmax = max(d);
delta = (dmax-dmin)/(L-1);
index = round((d-dmin)/delta);
index(index<0) = 0;
index(index>L-1) = L-1;
dq = dmin + index*delta;

% Encoding
bits = ceil(log2(L));
disp('Sample   Difference   Quantized Difference   Binary Code')
for i = 1:length(index)
    code = dec2bin(index(i), bits);
```

```

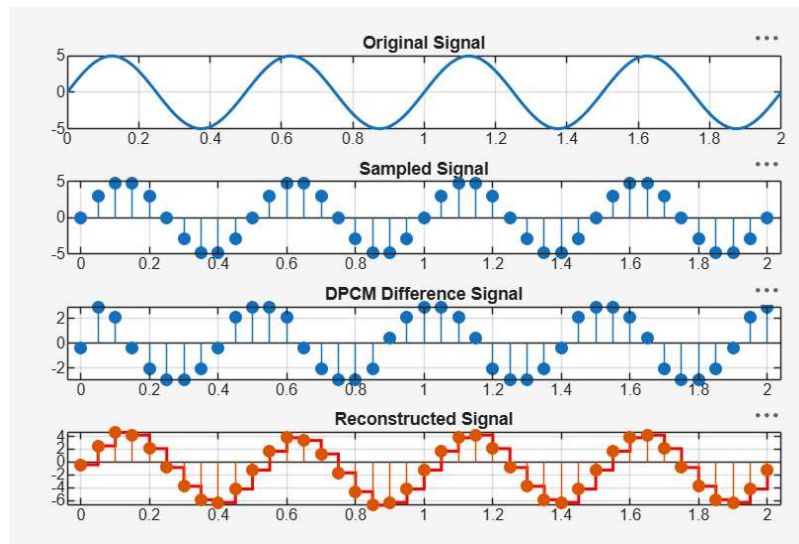
        fprintf('%2d\t %8.2f\t\t %8.2f\t\t %s\n', i, d(i), dq(i), code);
    end

% Reconstruction
xr = zeros(size(xs));
for i = 1:length(xs)
    if i == 1
        xr(i) = dq(i);
    else
        xr(i) = xr(i-1) + dq(i);
    end
end

% Plot
figure
subplot(4,1,1); plot(t,x,'LineWidth',1.5); title('Original Signal'); grid on
subplot(4,1,2); stem(ts,xs,'filled'); title('Sampled Signal'); grid on
subplot(4,1,3); stem(ts,dq,'filled'); title('DPCM Difference Signal'); grid on
subplot(4,1,4); stairs(ts,xr,'r','LineWidth',1.5); hold on
stem(ts,xr,'filled'); title('Reconstructed Signal'); grid on

```

MATLAB Output:



Simulink Implementation:

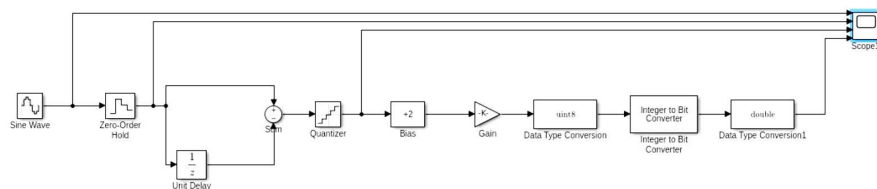
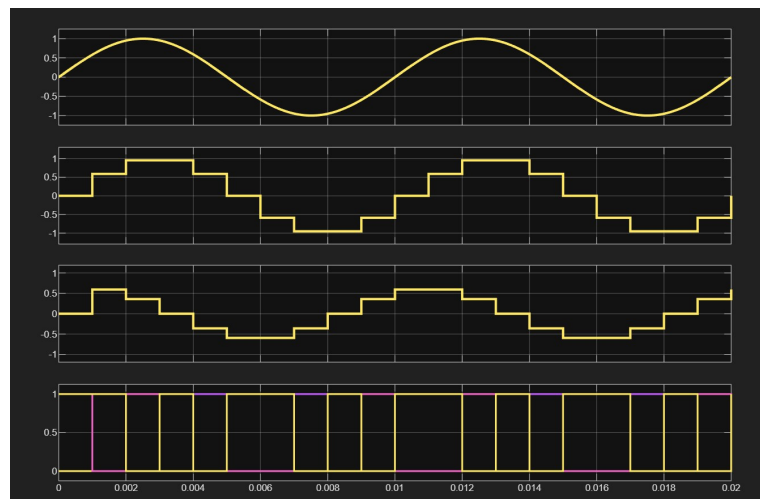


Fig: Differential Pulse Code Modulation

Simulink Output:



Observation:

- The DPCM difference signal has a much smaller amplitude range than the original sampled signal, since it represents only the change between consecutive samples.
- The reconstructed signal, obtained by cumulatively adding the quantized differences, closely follows the shape of the original sampled signal.
- The Simulink model uses a Unit Delay block to generate the predicted sample, a Sum block to compute the prediction error, and Quantizer, Bias and Gain blocks to encode the difference signal.

Result:

Differential Pulse Code Modulation was successfully simulated and implemented using both MATLAB and Simulink. The prediction error was quantized, encoded, and used to reconstruct a signal that closely approximates the original waveform, confirming the bit-rate advantage of DPCM over conventional PCM.

EXPERIMENT 7

Objective:

To simulate and implement Delta Modulation (DM) using MATLAB and Simulink.

Prerequisites:

- Set up MATLAB software.
- Set up Simulink.
- Basic knowledge of differential coding techniques.

Theory:

Delta Modulation (DM) is a digital modulation technique and a simplified form of Differential Pulse Code Modulation (DPCM). In DM, instead of transmitting the actual amplitude of each sample, only the change in amplitude between the present sample and the previous reconstructed sample is transmitted. Therefore, only one bit is required for each sample.

In the transmitter, the input analog signal is first sampled at a sufficiently high sampling rate. Each sampled value is compared with the previous reconstructed value. If the present sample is greater than the previous reconstructed value, a 1 is generated and the reconstructed signal is increased by a fixed step size Δ . If the present sample is smaller, a 0 is generated and the reconstructed signal is decreased by Δ . This produces a staircase approximation of the input signal, which can be expressed as:

$$x_r(n) = x_r(n-1) + \Delta b(n)$$

where $b(n)$ represents the transmitted bit and determines whether the staircase moves upward or downward.

At the receiver, the same step size is used to reconstruct the signal from the received binary sequence. The quality of reconstruction depends mainly on the sampling frequency and step size. If the step size is too small compared with the rate of change of the input signal, slope overload distortion occurs. If the step size is too large, the reconstructed signal fluctuates around the input signal, producing granular noise.

The major advantages of Delta Modulation are its simple implementation, low bit rate, and reduced circuit complexity, since only one bit is transmitted per sample. However, it is more susceptible to slope overload and granular noise compared with more advanced differential coding techniques.

MATLAB

```
clc; clear; close all;

% Continuous signal
A = 5; f = 2;
t = 0:0.001:2;
x = A*sin(2*pi*f*t);

% Sampling
Fs = 20;
ts = 0:1/Fs:2;
xs = A*sin(2*pi*f*ts);

% Delta Modulation
delta = 1.5; % Step size
dm = zeros(size(xs));
bits = zeros(size(xs));
dm(1) = 0;

for i = 2:length(xs)
    if xs(i) >= dm(i-1)
        bits(i) = 1;
        dm(i) = dm(i-1) + delta;
    else
        bits(i) = 0;
```



```

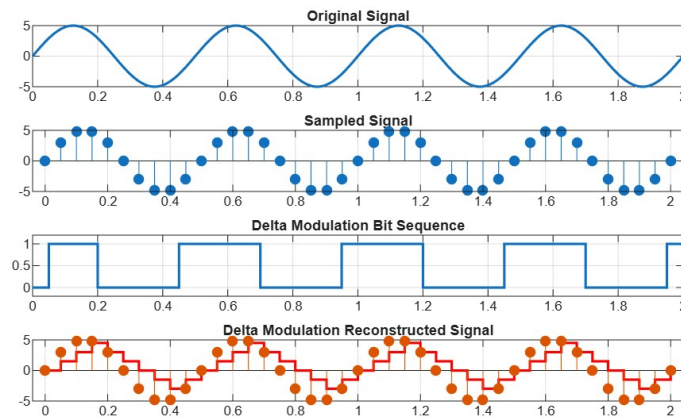
        dm(i) = dm(i-1) - delta;
    end
end

% Display encoded bits
disp('Sample  Input Value  DM Bit  Reconstructed Value')
for i = 1:length(xs)
    fprintf('%2d\t %8.2f\t %d\t\t %8.2f\n', i, xs(i), bits(i), dm(i));
end

% Plot
figure
subplot(4,1,1); plot(t,x,'LineWidth',1.5); title('Original Signal'); grid on
subplot(4,1,2); stem(ts,xs,'filled'); title('Sampled Signal'); grid on
subplot(4,1,3); stairs(ts,bits,'LineWidth',1.5); ylim([-0.2 1.2])
title('Delta Modulation Bit Sequence'); grid on
subplot(4,1,4); stairs(ts,dm,'r','LineWidth',1.5); hold on
stem(ts,xs,'filled'); title('Delta Modulation Reconstructed Signal'); grid on

```

Output:



Simulink Implementation:

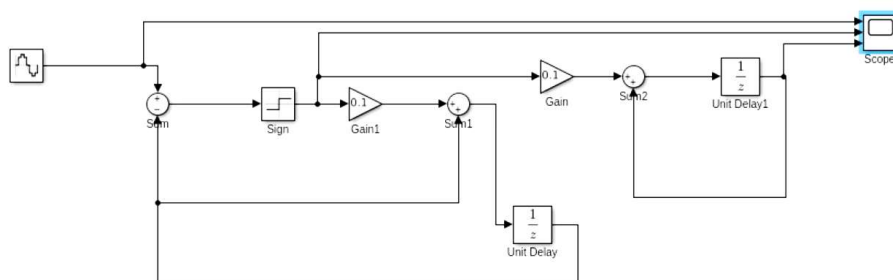
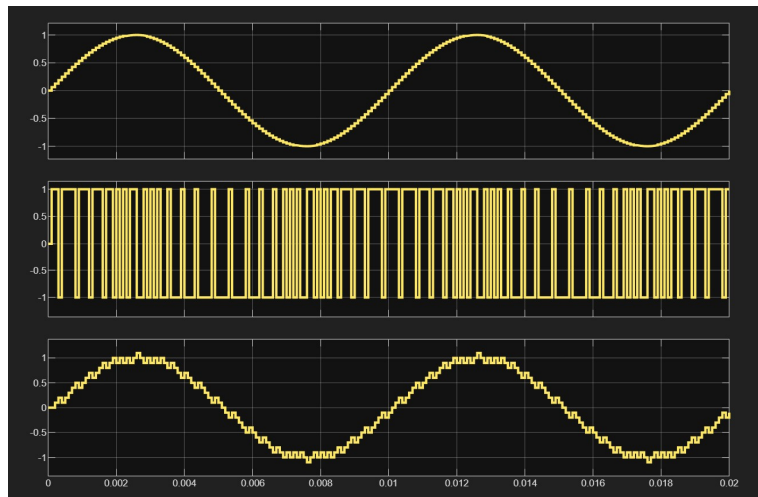


Fig: Delta Modulation

Simulink Output:



Observation:

- The Delta Modulation bit sequence generates a 1 whenever the sampled signal rises above the previous reconstructed value, and a 0 otherwise.
- The reconstructed staircase signal follows the general trend of the original sinusoid, but with visible step-wise approximation due to the fixed step size.
- Since only one bit is transmitted per sample, Delta Modulation achieves a lower bit rate than PCM and DPCM, at the cost of coarser signal reconstruction.

Result:

Delta Modulation was successfully simulated and implemented using both MATLAB and Simulink. The one-bit-per-sample encoded sequence was used to reconstruct a staircase approximation of the original sinusoidal signal, verifying the basic principle of Delta Modulation.

EXPERIMENT 8

Objective:

To simulate and implement Adaptive Delta Modulation (ADM) using MATLAB and Simulink.

Prerequisites:

- Set up MATLAB software.
- Set up Simulink.
- Basic knowledge of Delta Modulation and its limitations.

Theory:

Adaptive Delta Modulation (ADM) is an improved form of Delta Modulation in which the step size Δ is varied according to the rate of change of the input signal. Unlike conventional Delta Modulation, which uses a fixed step size, ADM automatically increases or decreases the step size to obtain better signal tracking.

In ADM, the input signal is sampled and compared with the previous reconstructed value. A 1 is generated when the input is greater than the reconstructed signal, while a 0 is generated when it is smaller. When several consecutive bits indicate that the signal is changing rapidly in the same direction, the step size is increased, allowing the reconstructed staircase signal to follow rapid changes and reducing slope overload distortion. When the signal changes slowly or the direction of change alternates, the step size is decreased, reducing granular noise.

The reconstructed signal can be represented as:

$$x_r(n) = x_r(n-1) + \Delta(n) b(n)$$

where $\Delta(n)$ is the adaptive step size and $b(n)$ determines the direction of the step.

The main advantage of ADM is that it provides better signal tracking than conventional Delta Modulation by adapting the step size to the input signal. It reduces both slope overload and granular noise over a wider range of signal variations, while retaining the simple one-bit-per-sample nature of Delta Modulation.

MATLAB

```
clc; clear; close all;

% Continuous signal
A = 5; f = 2;
t = 0:0.001:2;
x = A*sin(2*pi*f*t);

% Sampling
Fs = 20;
ts = 0:1/Fs:2;
xs = A*sin(2*pi*f*ts);

% Adaptive Delta Modulation
delta = 0.5; % Initial step size
delta_min = 0.1;
delta_max = 2;
dm = zeros(size(xs));
bits = zeros(size(xs));
step = zeros(size(xs));
step(1) = delta;

for i = 2:length(xs)
    if xs(i) >= dm(i-1)
        bits(i) = 1;
        dm(i) = dm(i-1) + step(i-1);
    else
        bits(i) = 0;
```

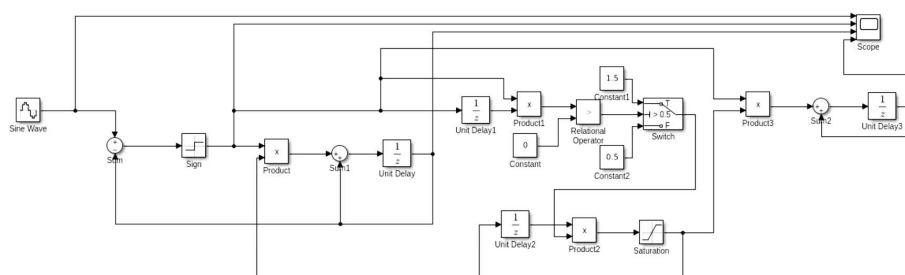
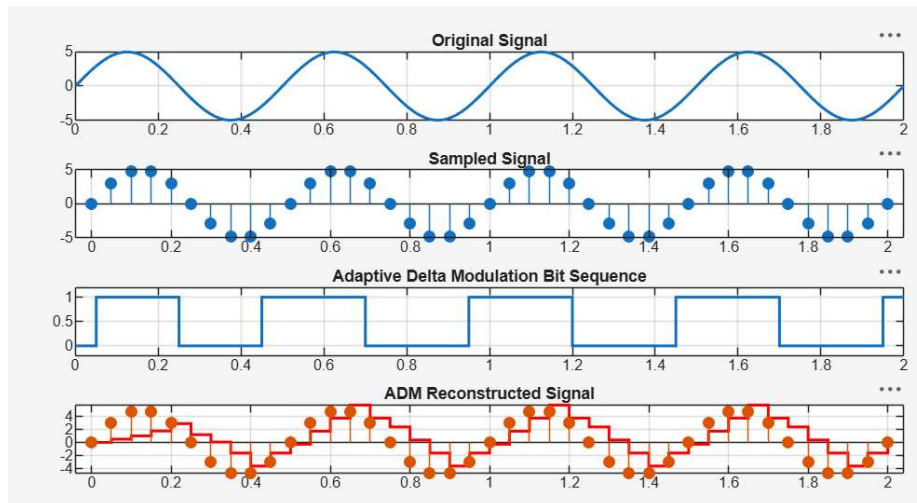
```

dm(i) = dm(i-1) - step(i-1);
end
% Adapt step size
if i > 2
    if bits(i) == bits(i-1)
        step(i) = min(step(i-1)*1.5, delta_max);
    else
        step(i) = max(step(i-1)/1.5, delta_min);
    end
else
    step(i) = step(i-1);
end
end

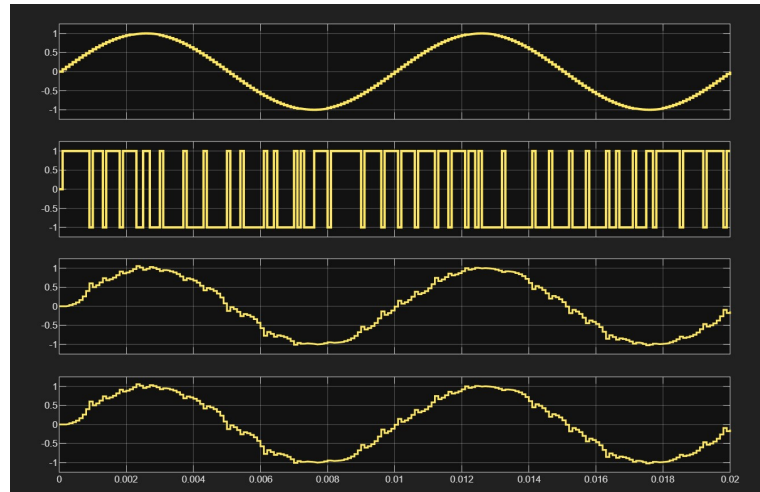
% Display encoded bits
disp('Sample  Input Value  ADM Bit  Step Size  Reconstructed Value')
for i = 1:length(xs)
    fprintf('%2d\t %8.2f\t %d\t\t %6.2f\t\t %8.2f\n', ...
        i, xs(i), bits(i), step(i), dm(i));
end

% Plot
figure
subplot(4,1,1); plot(t,x,'LineWidth',1.5); title('Original Signal'); grid on
subplot(4,1,2); stem(ts,xs,'filled'); title('Sampled Signal'); grid on
subplot(4,1,3); stairs(ts,bits,'LineWidth',1.5); ylim([-0.2 1.2])
title('Adaptive Delta Modulation Bit Sequence'); grid on
subplot(4,1,4); stairs(ts,dm,'r','LineWidth',1.5); hold on
stem(ts,xs,'filled'); title('ADM Reconstructed Signal'); grid on

```



Simulink Output:



Observation:

- The step size increases automatically during periods of rapid signal change (consecutive identical bits) and decreases when the bit direction alternates frequently.
- The ADM reconstructed signal tracks the original sinusoid more smoothly than conventional Delta Modulation, with reduced slope overload near the steep parts of the waveform.
- The Simulink model implements the adaptive step-size logic using Relational Operator, Switch, Product and Unit Delay blocks in a feedback configuration.

Result:

Adaptive Delta Modulation was successfully simulated and implemented using both MATLAB and Simulink. The variable step size improved the tracking of the reconstructed signal compared to conventional Delta Modulation, confirming the advantage of ADM in reducing slope overload and granular noise.